Reviewer Pack — Hodge Theoretic Complexity of Boolean Function Entropy

Entry 5d76cae93c8b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 23:45:07 UTC

Hodge Theoretic Complexity of Boolean Function Entropy

Entry ID: `5d76cae93c8b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-30 23:45:07 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (specifically, Hodge Theory) **Field B** (complexity object): Boolean function entropy

Statement:

> The Hodge number h^{1,0} of the moduli space of complex curves associated with a Boolean function φ: {0,...,n-1} -> {0,1} is upper bounded by the entropy of φ, i.e., h^{1,0}(φ) ≤ Entropy(φ), where Entropy(φ) = -Σ_{x in support(φ)} P(x) log P(x).

Rationale (proposer's reasoning):

> Hodge theory provides a way to study the algebraic and geometric properties of complex curves. By relating this to the entropy of Boolean functions, we may uncover new insights into the complexity of computation.

Taxonomy category: `HODGE_THEORY_ENTROPY_BOUND` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `574b4e6ac89918d4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all 30 randomly generated Boolean functions, the Hodge number h^{1,0} is less than or equal to the entropy by at least 95% of the time.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge Theory" AND "Boolean function entropy"` - `"moduli space of complex curves" AND entropy AND algebraic geometry` - `algebraic geometry intersection Boolean function entropy`

Top relevant hits considered: - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/2204.10334v1] Machine Learning Algebraic Geometry for Physics

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def entropy(phi):
        total = sum(phi.values())
        return -sum(Fraction(count, total) * math.log2(Fraction(count, total)) for count in phi.va

    def hodge_number(phi):
        n = len(phi)
        support = list(phi.keys())
        P = [Fraction(phi[x], n) for x in support]

        # Construct the associated complex curve
        points = [(x // (2 ** i)) % 2 for i in range(n)]
        matrix = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n):
            matrix[i][i] = 1
            for j in range(i + 1, n + 1):
                if points[j - 1] == points[i]:
                    matrix[i][j] = 1

        # Gaussian elimination to find the rank of the matrix
        def gaussian_elimination(A):
            m, n = len(A), len(A[0])
            for i in range(m):
                max_row = i
                for j in range(i + 1, m):
                    if abs(A[j][i]) > abs(A[max_row][i]):
                        max_row = j
                A[i], A[max_row] = A[max_row], A[i]
```

```
                if A[i][i] == 0:
                    continue
                for j in range(n):
                    A[i][j] /= A[i][i]
                for k in range(m):
                    if k != i and A[k][i] != 0:
                        factor = A[k][i]
                        for j in range(n):
                            A[k][j] -= factor * A[i][j]
            return sum(1 for row in A if any(row[j] != 0 for j in range(n)))

        rank = gaussian_elimination(matrix)
        return n - rank

    def generate_boolean_function(n):
        return {x: random.choice([0, 1]) for x in range(n)}

    results = []
    for _ in range(30):
        phi = generate_boolean_function(random.randint(5, 40))
        hodge = hodge_number(phi)
        ent = entropy(phi)
        results.append((hodge, ent))

    mean_hodge = sum(h for h, _ in results) / len(results)
    mean_entropy = sum(e for _, e in results) / len(results)
    support_fraction = sum(1 for h, e in results if h <= e) / len(results)

    return {
        "metric_name": "Hodge number vs Entropy",
        "metric_value": mean_hodge,
        "instances_tested": 30,
        "n_max": max(len(phi) for phi, _ in results),
        "conjecture_holds": support_fraction >= 0.95,
        "counterexample": "" if support_fraction >= 0.95 else f"H^(1, 0)({phi}) = {mean_hodge}, Er
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(2, 6)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = (sum((x - mean)**2 for x in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r >= 0.95 * max(results)) / len(results)

    if all(r >= 0.95 * max(results) for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(r < 0.95 * max(results) for r in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if result < 0.95 * m
        print(f"RESULT: FALSIFIED counterexample=\"H^(1, 0) > Entropy\" first_failing_seed={first_
```

```
    else:
        print("RESULT: INCONCLUSIVE insufficient support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94d6c9ec.py", line 91, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94d6c9ec.py", line 68, in run_trial
    hodge = hodge_number(phi)
            ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94d6c9ec.py", line 31, in hodge_number
    points = [(x // (2 ** i)) % 2 for i in range(n)]
               ^
UnboundLocalError: cannot access local variable 'x' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15017
2	preregistration	ollama_remote	glm4:latest	0	0	27372
3	novelty	ollama_remote	glm4:latest	0	0	10755
4	novelty	ollama_remote	glm4:latest	0	0	15659
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13465
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37338
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17891
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12432
9	judge	ollama_remote	glm4:latest	0	0	9626

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 159555 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5d76cae93c8b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5d76cae93c8b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5d76cae93c8b.tar.gz` (if generated)